13. Scaling Laws

19 March 2026 14:14

Basics - Cross entropy measures the performance of a classification model
logarithmic loss difference between predicted prob distn and actual class labels

binary cross entropy - for two class problems

Categorical cross entropy for multi class classification where each input belongs to exactly one class

Shannon entropy eqn $H = -\sum p(x) \log p(x)$

Cross entropy measures the average number of bits required to identify an event from one probability distribution p using the optimal code for another probability distribution q

measures the difference between the observed probability distribution of a classification model & predicted values

What's the diff b/w this & KL divergence?
↳ that avg no of bits line from above while KL quantifies amount of information loss when Q is used to approximate P

avg cross-entropy across all data samples

binary cross entropy loss $L = -\frac{1}{N}\left[\sum_{i=1}^{N} [(t_i)\log(p_i) + (1-t_i)\log(1-p_i)]\right]$

$N \to$ data points
t_i the truth value taking a value 0 or 1
p_i softmax probability for the ith data point

The loss scales as a power law with model size, dataset size and the amount of compute used for training

simple equations govern the dependence of overfitting on model/dataset size and the dependence of training speed on model size

These equations allow us to determine the optimal allocation of a fixed compute budget

Larger models → significantly more sample efficient

Optimally compute efficient training involves training very large models on a relatively modest amount of data and stopping significantly before convergence

This work observes precise power law scalings for performance as a function of training time, context length, dataset size, model size and compute budget

Core variables
$N \to$ parameters
$D \to$ dataset size
$C \to$ compute
$L \to$ Loss

Loss follows power scaling with C, D, N

Performance scales as one of the above grows but when not bottlenecked by other two

N-limited (capacity bottleneck)
D-limited (data bottleneck)
C-limited (computation / optimization bottleneck)

Performance doesn't go down but saturates when other two become a bottleneck

eg
$N \uparrow$ D fixed → model memorizes → no generalization improvement
$D \downarrow$ N fixed → model capacity insufficient

Larger models are more sample efficient
↳ They reach same loss with fewer steps & tokens
But each step is more expensive

Penalty = Extra loss due to imbalance (How far are you from ideal case)

formally $\text{Penalty} = L(N,D) - L(N,\infty)$

To avoid penalty scale $D \propto N^{0.74}$ $D \propto N^{\alpha_N/\alpha_D}$ → Scaling constants

If N grows faster than D the model starts fits

$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}$ $N \to$ actual model size, $N_c \to$ reference scale (fitted const)

N_c sets the horizontal position of the curve
α controls the slope

$L = f(\min(\text{capacity}, \text{data}, \text{compute}))$

Scaling one dimension alone is useless beyond a point

N_c, C_c, D_c have no fundamental meaning → They depend tokenization, dataset, setup

Only α (exponents) are fundamental

It's C, D, N all the time

Everything else architecture, optimizer, tricks } secondary

Here all equations are

$$L \propto \frac{1}{X^{\alpha}} \quad X \in (N, D, C)$$

Increase X loss decreases but with diminishing returns

Batch size is tied to optimization efficiency

It scales with loss

$L(N) \propto N^{-\alpha_N}$ $\alpha_N \approx 0.076$ $N_c \approx 8.8 \times 10^{12}$

$L(N,D) = \left[\left(\frac{N_c}{N}\right)^{\alpha_N} + \frac{D_c}{D}\right]^{\alpha_D}$ → loss controlled by two competing bottlenecks

Capacity term, data term → if N is small → first term dominates, if D is small → second term dominates

$D \propto N^{0.74}$

Summary of scaling laws

Optimally allocated compute budget C_{min}

$\frac{1}{\text{term}}$ term if D is small → second term dominates

$D \propto N^{0.74}$

Summary of scaling laws

Transformers

test loss can be predicted using a power law when performance is limited by only C D N → no of non embedding parameters

C: Optimally allocated compute budget C_{min}

D: dataset size

1 For models with a limited number of parameters, trained to convergence on sufficiently large datasets

$L(N) = (N_c/N)^{\alpha_N}$, $\alpha_N \sim 0.076$, $N_c \sim 8.8 \times 10^{13}$ (non embedding params)

2 For large models trained with a limited dataset with early stopping

$L(D) = (D_c/D)^{\alpha_D}$ $\alpha_D \sim 0.095$ $D_c \sim 5.4 \times 10^{13}$ tokens

3 When training with a limited amount of compute, a sufficiently large dataset, an optimally sized model, and a sufficiently small batch size (making optimal use of compute)

$L(C_{min}) = \left(C_c^{min}/C_{min}\right)^{\alpha_C^{min}}$, $\alpha_C^{min} \sim 0.050$, $C_c^{min} \sim 3.1 \times 10^8$ (PF-days)

The power laws α_N α_D α_C^{min} specify the degree of performance improvement expected as we scale up C D N

precise $C_c P_c N_c$ depends on vocabulary size, tokenization } No fundamental meaning

$L(N, D) \simeq$ function of $\frac{N^{0.74}}{D}$

N↑ ratio inc → overfitting↑ → model capacity >> data information

Generalization stops improving

Model memorizes

Loss is controlled by worst bottleneck

D↑ initially helps, model becomes capacity limited

Loss plateaus

Additional data gives no benefit

N↑ D fixed → data bottleneck

D↑ N fixed → capacity bottleneck

Model cannot represent complex patterns, even with more data

Perplexity

* cross entropy loss → not bounded by 1

loss > 1 higher uncertainty / worse predictions

$B_{crit}(L) = \frac{B_*}{L^{1/\alpha_B}}$

B_* → $\sim 2 \times 10^8$ tokens

$1/\alpha_B$ → ≈ 0.21 (scaling exponent)

L → current loss

→ opt batch size for convergence level

As loss↓ (model improves) → early training → noisy gradients

→ B_{crit} ↑

later training → gradients become subtle

↳ need larger batch to reduce noise

$L(N, S) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{S_c}{S_{min}}\right)^{\alpha_S}$

$S_c \approx 2.1 \times 10^3$

$\alpha_S \approx 0.76$

↳ capacity limit

→ Optimization limit

More steps → lower loss

Captures training progress

Batch size is not arbitrary

↳ it is determined by loss level

Training dynamics decomposition

Total error = approximation error + optimization error

Loss = capacity error + optimization error

if S small → undertrained → second term dominates

N small → under parameter → first term dominates

Compute Budget $C = 6NBS$

given fixed C how should we choose the trade off to maximize L"

$S \propto \frac{C}{NB}$

$L(N, S) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{S_c}{S}\right)^{\alpha_S}$

$L = N^{-\alpha_N} + \left(\frac{NB}{C}\right)^{\alpha_S}$

Training is a 3 way trade off

Model size (representation)

steps (optimization)

Batch size (parallelism)

optimizing above gives

$N \propto C^{\alpha_C^{min}/\alpha_N}$, $B \propto C^{\alpha_C^{min}/\alpha_B}$, $S \propto C^{\alpha_C^{min}/\alpha_S}$ → defines optimal resource allocation

$\alpha_C = 1/\left(1/\alpha_S + 1/\alpha_B + 1/\alpha_N\right)$ → combined scaling exp

optimization, batch/parallelism, capacity

Compute scaling is limited by all three processes simultaneously

$N \propto C_{min}^{0.73}$, $B \propto C_{min}^{0.24}$, $S \propto C_{min}^{0.03}$

Model size grows very fast

Batch size grows moderately

step grows almost not at all

$L = N^{-\alpha_N} + S^{-\alpha_S}$

capacity, optimization

Training longer is inefficient

Making model bigger is efficient

As model improves → you can increase batch size

So instead of increasing steps (S)

↳ increase batch size (B)

Process more data in parallel

↑N improves loss slowly but consistently

↑S improves loss faster initially, then saturates

$S^{-\alpha_S}$ $\alpha_S \approx 0.76$ large exponent → fast diminishing returns

After some steps → almost no improvement

$N^{-\alpha_N}$ $\alpha_N \approx 0.07$ small exponent → slow but steady gains

Keeps improving even at large scale

So why B depends on L ? Signal to noise ratio of gradients depends on how close you are to —

Definition of model size (N)

$N \approx 12\, n_{layer} \times d_{model}^2$

Model size grows as depth × width²

When they say N they are changing width (d_{model}), layers (n_{layer}) effectively

They exclude embedding parameters

↓

don't behave like core model capacity

$C \approx 6NBS$ → compute = model size × data processed

Parameters

Model size $N \sim d^2 \times$ layer

Compute $C \sim NBS$

Data seen $D \sim BS$

Increase N compute ↑

Increase B few steps needed

Increase S more compute

factor 6 = forward pass + backward pass

forward pass cost $C_{forward} \approx 2N + 2\, n_{layer}\, n_{ctx} \times d_{model}$

Dominated by N

Compute scales by model size, not sequence length

in experiments they varied

i model size

ii Dataset size

iii Shape

iv Context length

v Batch size

embedding parameters → parameters used to convert tokens → vectors

↳ token + positional

non-embedding params → all parameters inside the transformer layer

↳ include attention and ff layers, output projections

$N = 12\, n_{layer}\, d_{model}$

Performance mainly depends on attention + ff layers

Transformer performance depends very weakly on the shape parameters n_{layer}, n_{heads} and d_{ff} when we hold N fixed

they aren't considered really because

i They don't scale the same way

Embedding size ~ v · d_{model}

Transformer size ~ d_{model}^2

Different scaling behaviour

iii Including embeddings breaks clean scaling laws

Embedding matrix can make smaller without impacting

ii Embeddings aren't the bottle neck

they store representations, don't perform deep transformations

Embedding params

Scale linearly with vocab

don't increase expressivity as much

→ comparing LSTM & transformer performance w.r.t N

↳ perform as well as transformers for tokens appearing early in the context but cannot match on later tokens

Transformer params

increase quadratically

increase representational power slightly

power law relation between performance and context position → increasingly large powers for larger models suggest improved ability to quickly recognize patterns

→ Generalization depends almost exclusively on the in-distribution validation loss, and does not depend on the duration of training or proximity to convergence

no dependence on model depth

At fixed compute budget (N), performance is nearly invariant to architecture shape

$C = 6NBS$?

extreme shapes (too shallow/too deep) → deviate

< 2 layers → worse

Generalization = shifted version of training loss

$L_{OOD} \approx L_{train} + \text{constant}$

improve training loss → generalization improves automatically

for a fixed C $\{S = \frac{C}{6NB}\}$

increases N → reduces S

large models need → few steps, few tokens

N transformer ~ d_{model}^2

Context computation/reasoning drives performance scaling

$N_{embed} \sim v \cdot d_{model}$

Scale linearly

Depend on vocab

If $N_{total} = N_{embed} + N_{transformer}$

for small models → embeddings dominate

for large models → transformer dominates

Compute ← forward pass (2N)

predictions each param is involved in two operations → a multiplication and one addition to calculate layer output

$C_{forward} \approx 2N$

↓

multiply + add → 2 FLOPs per param

hence 2N operations per token

Compute gradients + Update weights ← Backward pass (4N)

↓

performs two distinct matrix multiplications of equal size to forward pass

Backward pass costs ~ 2× forward

$C_{backward} \approx 4N$

↳ contributes FLOPs of forward because it must perform two distinct gradient calculations for every one calculation in forward pass

Gradients w.r.t activations (δh)

Calculating error signal to propagate back to the previous layer

Gradients w.r.t weights (G)

Calculating how much each weight needs to change per update

4. L(N,D) equation

capacity limit

$$\left[\left(\frac{N_c}{N_D}\right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D}\right]^{\alpha_D}$$

data limit

parameterization using three principles

1 changes in Vocabularization are expected to rescale the loss by an overall factor. The parameterization of L(N,D) must naturally allow for such a rescaling

2 Fixing D, sending $N \to \infty$ overall loss should approach L(D) & vice versa

3 L(N,D) should be analytic at $D = \infty$ so that it has a series expansion in 1/D with integer powers

Everything related to signal to noise ratio or variance and scales as 1/D

finite D to infinite D limit depending of as

$$\delta L(N,D) \equiv \frac{L(N,D)}{L(N,\infty)} - 1$$

$$\delta L \approx \left(1 + \left(\frac{N}{N_c}\right)^{\frac{\alpha_N}{\alpha_D}} \frac{D_c}{D}\right)^{\alpha_D} - 1$$

at large D this formula also has a series expansion in powers of 1/D

$$D = (5 \times 10^2) N^{0.74}$$

Overfitting, imbalance betn N & D ratio $\frac{N^{0.74}}{D}$ controls behaviour

they say Overfitting ~ variance ~ D

Data noise behaves like statistical variance

↳ scales as 1/D

$D \geq (5\times10^{3})\, N^{0.74}$

Overfitting: imbalance betn N & D rates $\frac{N^{0.74}}{D}$ controls behaviour

they say Overfitting ~ variance ~ D

Data set is a sample from true distⁿ, Finite D → sampling noise

Variance $\propto \frac{1}{D}$ More data has less noise → less overfitting

hence $\delta L \propto \frac{1}{D}$

in eq $\left(\frac{N_c}{N}\right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D_N}$ → N enters with exponent ratio, D with linear term

They fit/fig → writing expansion in powers of 1/

as $D \to \infty$

$L(N,D) = L(N) + \frac{\text{corrections}}{D} + \frac{\text{higher terms}}{D^2}$

this requires D to behave like 1/D

Data noise behaves like statistical variance → scales as 1/D

N doesn't → the asymmetry in the equation

$\delta L = \frac{L(N,D)}{L(N,\infty)} - 1$ → how much worse are you due to finite data

Minimum data required to avoid overfitting

$D \geq 5\times10^{3}\, N^{0.74}$

$\delta L \sim \frac{N^{0.74}}{D}$ if ratio small → no overfitting, if ratio large → overfitting

Overfitting grows when model capacity grows faster than available data

5 Scaling laws with Model size and Training time

Adjustment for training at $B_{crit}(L)$

for B upto B_{crit}, the batch size B can be increased with very minimal degradation in compute efficiency

$B > B_{crit}$ → increase in B returns diminishing results

To utilize training time & compute effectively, it is best to train with a batch $B \approx B_{crit}$

training at $B > B_{crit}$ reduces no of steps, while $B \ll B_{crit}$ minimizes the use of compute

for no of training steps S and the no of data examples processed $E = BS$ satisfy

$$\left(\frac{S}{S_{min}} - 1\right)\left(\frac{E}{E_{min}} - 1\right) = 1$$ when training to fix any value of loss L

trade off betn steps (time), data processed (compute)

S_{min} min of steps to reach L

$B_{crit}(L) \equiv \frac{E_{min}}{S_{min}}$ → optimal balance between parallelism (B) learning (S)

→ independent of model size

→ depends only on L

reduce steps → increase data, reduce data → increase steps } trade

$B_{crit}(L) \approx \frac{B_*}{L^{1/\alpha_B}}$ $\alpha_B = 0.21$, $B_* \approx 2\times10$

Best to estimate relation betn S while training at $B = 2^{12}$ tokens and no of training steps while $B \gg B_{crit}$

$S_{min}(S) = \frac{S}{1 + B_{crit}(L)/B}$ (min steps at $B \gg B$) → increasing B reduces steps (but inefficiently)

Thus also defines a critical value of compute needed to train L with a model of size N if we were to train $B \ll B_{crit}(L)$

$C_{min}(C) = \frac{C}{1 + B/B_{crit}(L)}$ (min compute, $B \ll B_{crit}$) → increasing B reduces compute cost

Where $C = 6NBS$

$S = \frac{C}{6NB}$ if N goes too high: S goes too low, Model undertrained, optimization term ↑, Loss increases

$L(N, S_{min}) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{S_c}{S_{min}}\right)^{\alpha_S}$

Lower bound on early stopping step

$S_{stop}(N,D) \gtrsim \frac{S_c}{(L(N,D) - L(N,\infty))^{1/\alpha_S}}$

→ stop when optimization error ≈ data limitation

if you train longer → only fitting noise, no real improvement

Stop when further training only reduces overfitting gap

Too big model + fixed compute = undertraining

two different optimization: reduce compute, reduce steps

Pointers

$B < B_{crit}$: gradients noisy, Increase B → reduce noise → improves efficiency, need more steps S

$B > B_{crit}$: Gradients already low noise, B inc ↑ doesn't improve learning only cost/step, Waste of compute

As loss reduces gradients become smaller, need larger batch to estimate them accurately, $L \downarrow \Rightarrow B_{crit} \uparrow$

$L = 6SBN$

$N(C_{min}) \propto (C_{min})^{0.72}$ $C_{min} = 6NB_{crit}S$

$B_{crit} \propto (C_{min})^{0.24}$

$S_{min} \propto (C_{min})^{0.03}$

As we scale up language modeling with an optimal allocation of computation, we should predominantly increase the model size N while simultaneously scaling up batch size via $B \propto B_{crit}$ with negligible increase in number of steps

$L(N, S_{min})$ substitute $S_{min} = \frac{C_{min}}{6NB}$, find min as a function of N, while fixing the training compute

$L(C_{min}) = \left(\frac{C_c^{min}}{C_{min}}\right)^{\alpha_C^{min}}$ $\alpha_C^{min} = \frac{1}{1/\alpha_S + 1/\alpha_B + 1/\alpha_N} \approx 0.054$

$N(C_{min}) \propto (C_{min})^{\alpha_C^{min}} \approx (C_{min})^{0.71}$

Slow growth of data needed for compute efficient training

compute efficiency is limited by all these bottlenecks

To keep overfitting under control we should scale the data set size as

$D \propto N^{0.74} \propto C_{min}^{0.54}$

comparing this to data req of compute efficient training if $C = 2C_{min}$ (GPU-hours) and we never reuse data during training

$$D(C_{min}) = \frac{2C_{min}}{6N(C_{min})} \approx (4\times10^{10} \text{ tokens}) \left(C_{min}/\text{PF-day}\right)^{0.26}$$

This is the max rate at which dataset can grow productively with compute but we are only training for one epoch

Compute efficient training will eventually run into a problem with overfitting, even if training process never re-uses any data!

generally $L(D) \propto D^{-0.095}$

loss would scale with compute as $L(D(C_{min})) \propto C_{min}^{-0.026}$ since we are data limited

this will contradict $L(C_{min}) \propto C_{min}^{-0.050}$

the intersection point of $L(D(C_{min}))$ and $L(C)$

$C^* \sim 10^4$ PF days $N^* \sim 10^{12}$ params $D^* \sim 10^{12}$ tokens $L^* \sim 1.7$ nats/token

given C there is an optimal N

$L(N,S) = N^{-\alpha N} + S^{-\alpha S}$

$C = 6NBS$

optimize loss under fixed compute

$N_{opt}(C) \propto C^{0.73}$

$S_{opt}(C) \propto C^{0.03}$

$B_{opt}(C) \propto C^{0.24}$

Most compute increase N

Slightly increase B

Almost no increase S

Clean compute scaling appears only when training is critical

If model size isn't optimal you waste compute

Too small N → under capacity

Too large N → under trained

Both require more compute to reach same loss

There exists a sweet spot N^* for each C

Law 1 $L(C_{min}) \propto C^{-0.05}$ compute efficient training

Law 2 $D \propto N^{0.74}$

Since $N \propto C^{0.73}$

$D \propto C^{0.73 \times 0.74} \approx C^{0.54}$ → But actual data usage $D(C_{min}) \propto C^{0.26}$ (from training dynamics)

Req data ~ $C^{0.54}$

Available data used ~ $C^{0.2}$

Data grows too slowly → eventually training becomes data limited

So $L(C_{min})$ prediction breaks

Scaling laws are only valid until data becomes limiting

→ robust across datasets and architectures

No sharp jumps - just gradual improvement in performance

Architecture matters less than scale

Compute optimal training is critical

naive training → inefficient

optimal allocation → predictable scaling

$L = f(C, D, N)$ Optimal training

$L(N) \sim N^{-0.076}$ → Increase N aggressively

$L(D) \sim D^{-0.095}$ → Increase B moderately

$L(C) \sim C^{-0.15}$ → barely increase S

Limitation Eventually, data becomes bottleneck